

Exploiting Booster Pass Chain for Compiler Phase Ordering

Yihan Chen
Nanjing University
Nanjing, Jiangsu, China
yihanchen@smail.nju.edu.cn

Huanhuan Chen
Nanjing University
Nanjing, Jiangsu, China
522024330002@smail.nju.edu.cn

Yuan Yao
Nanjing University
Nanjing, Jiangsu, China
y.yao@nju.edu.cn

Ping Yu
Nanjing University
Nanjing, Jiangsu, China
yuping@nju.edu.cn

Feng Xu
Nanjing University
Nanjing, Jiangsu, China
xf@nju.edu.cn

Xiaoxing Ma
Nanjing University
Nanjing, Jiangsu, China
xxm@nju.edu.cn

Abstract

The phase ordering problem, which aims to find suitable pass sequences for a given program on a target architecture, is critical in compiler optimization. One key challenge of this problem lies in the complex interplay among different passes within the vast optimization space of possible pass sequences. To better explore the interplay among passes, this paper proposes a new concept called *booster pass chain* (BPC), and presents a novel approach that identifies and leverages the BPCs to optimize the code size. Specifically, a BPC is a sequence of passes with positive interplay that, when presented as a whole, may exhibit significant optimization effects for certain programs. We then propose an iterative algorithm to extract BPCs, based on which we build a candidate set of pass sequences. For a given program, we also train a neural network to predict the suitable pass sequences from the candidate set. Experimental evaluations on 16 datasets containing 6,186 programs demonstrate the effectiveness of the proposed approach. That is, the candidate set achieves an average of 9.9% improvement compared to the LLVM -Oz flag in code size reduction, and selecting the top-3 pass sequences using the neural network predictor achieves 6.9% improvement. Our code and results are available at <https://github.com/SoftWiser-group/EBPC4CPO>.

ACM Reference Format:

Yihan Chen, Huanhuan Chen, Yuan Yao, Ping Yu, Feng Xu, and Xiaoxing Ma. 2025. Exploiting Booster Pass Chain for Compiler Phase Ordering. In *the 16th International Conference on Internetware (Internetware 2025)*, June 20–22, 2025, Trondheim, Norway. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3755881.3755899>

1 Introduction

Phase ordering, i.e., determining the right pass sequence for a given program, is a key problem in compiler optimization [7, 38, 52]. It involves selecting and arranging the compiler passes to optimize the program. The complexity of phase ordering stems from the complex *interplay* among passes [1, 10, 35] within the huge optimization space of possible pass sequences [37, 50]. To this end, mainstream compilers have provided a set of standard pass sequences (such as -O3 and -Oz in LLVM), which generally perform well across a

large set of programs. However, such standard pass sequences may perform poorly for some programs, and it has been observed that customizing pass sequences for individual programs often yields better performance [6, 12, 37, 50].

A common way to address the phase ordering problem is called *iterative compilation* [10], which iteratively applies different pass sequences to explore the optimization space. Recently, iterative compilation tends to follow the *generate-and-select* framework, i.e., first generating a set of candidate pass sequences, followed by selecting sequences from this set [6, 12, 37, 50]. For example, the state-of-the-art method NVP [37] first employs a simple random search strategy in the optimization space to generate a *coreset* (i.e., a set of candidate pass sequences), and then trains a fancy neural network model to select pass sequences from this coreset for a given program. To simplify terminology, we adopt the term *coreset* in this work.

Orthogonal to NVP, our focus is on the *generate* side, i.e., to better explore the optimization space. To this end, various techniques have been studied, including random search [19, 37], genetic algorithms [15, 32, 50], greedy algorithms [28], and machine learning techniques [3, 5, 25, 29, 40]. However, none of them has explicitly modeled the *interplay* among different passes, which is crucial for the phase ordering problem [10]. For example, it has been observed that certain passes when combined together as a *pass chain*, are beneficial for a range of programs [6, 30, 38, 44]. A few methods [6, 38] have proposed to build pass sequences upon such beneficial pass chains; however, their exploration is restricted in a very limited space (e.g., the pass chains within standard pass sequences such as LLVM -O3). Given this, a natural question arises: *can we automatically identify and leverage the beneficial pass chains to better explore the optimization space?*

To answer the above question, this paper presents a novel phase ordering approach for the code size reduction task. The key behind our approach is the concept of *booster pass chain* (BPC), which assembles a few passes with positive interplay (i.e., strong optimization effects for certain programs) into a sequence. With a set of BPCs at hand, we can directly make use of them during the optimization space exploration.

We then follow the generate-and-select framework with special attention paid to the *generate* stage. Specifically, we propose an iterative algorithm to identify and leverage BPCs when constructing the coreset of candidate pass sequences. It forms a closed-loop of building pass sequences from BPCs, refining pass sequences, and extracting new BPCs from refined pass sequences. We develop a



This work is licensed under a Creative Commons Attribution 4.0 International License. *Internetware 2025, Trondheim, Norway*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1926-4/25/06
<https://doi.org/10.1145/3755881.3755899>

simple but effective greedy algorithm during pass sequence building and a backward-scan algorithm during BPC extraction, both helping to identify and leverage passes that have indirect but significant contributions. As to the *select* stage, we employ a simple multilayer perceptron (MLP) model to predict the appropriate pass sequences for a given program from our coreset.

To evaluate the effectiveness of the proposed approach on the code size reduction task, we use all the 14 built-in datasets from CompilerGym [20] (including Csmith [55], CLGen [18], MiBench [24], AnghaBench [21], etc.), as well as two additional datasets comprising user-submitted code from programming competition platforms. From the above 16 benchmarks, we randomly select 500 programs to construct the coreset in the generate stage, and 27,900 programs to train the MLP model in the select stage. We then test the performance of the proposed approach on the remaining 6,186 programs. The results show that, compared to the LLVM -Oz optimization level, our coreset achieves an average of 9.9% improvement in code size reduction (geometric mean). Additionally, using the MLP model to select top 3 pass sequences results in 6.9% improvement.

In summary, the main contributions of this paper include:

- The concept of booster pass chains for the phase ordering problem. The extracted BPCs can also be used for future compiler optimization research.
- An iterative approach to automatically identify BPCs and leverage them to generate a set of candidate pass sequences, which are potentially useful for a large range of programs.
- Extensive evaluations on 6,186 programs, showing significant improvements of the proposed approach over LLVM -Oz in terms of code size reduction.

2 Background and Motivation

2.1 Phase Ordering Problem

Compilation optimization aims to automatically transform the source code into the corresponding binary code that runs *better* on the target architecture. Here, “better” refers to the optimization target including code size, execution time, power consumption, etc. Modern compilers (e.g., GCC and LLVM) usually provide a set of compiler passes, where a compiler pass is a single traversal through the source code or its intermediate representation. To this end, two compiler optimization problems have attracted great research attention, i.e., *pass selection* and *phase ordering*. The former aims to select the best compiler passes for a given program [57], and the latter further takes into account the order to apply these passes [37].

In this work, we tackle the phase ordering problem, which is very challenging as the optimization space (i.e., the number of possible pass sequences) grows exponentially with the number of available passes. For example, the latest GCC and LLVM have more than 290 and 220 passes, respectively, meaning that there are billions of possible pass sequences even with a relatively short sequence length. To complicate things further, the passes are interdependent in the sense that applying one pass first may benefit the subsequent others [7]. For example, some passes may initially deteriorate the performance but achieve further improvement if combined with some other passes.

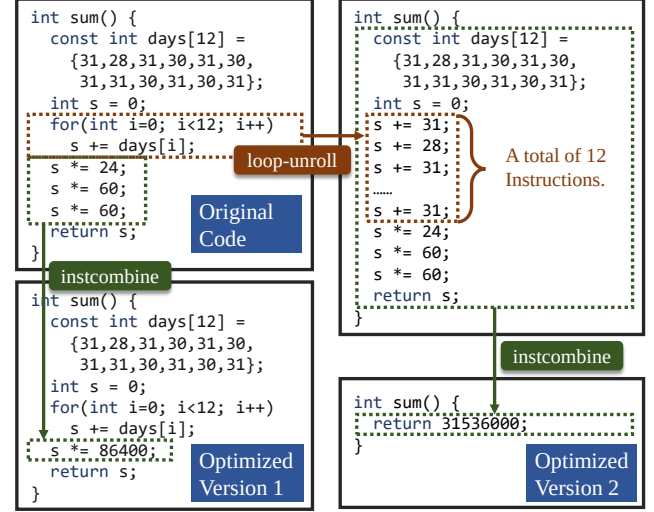


Figure 1: Motivation example. Applying `loop-unroll`, which first degrades the performance, before `instcombine` could further improve the optimization performance compared to applying `instcombine` alone. We name passes like `loop-unroll` as *assistant passes*, and pass chains like `[loop-unroll, instcombine]` as *booster pass chains*.

2.2 Motivation Example

Figure 1 shows an example code snippet that calculates the number of seconds in a year. To optimize this code snippet, a naive greedy strategy may choose the `instcombine` (instruction combine) pass to shrink the code size (optimized version 1 in the figure). However, if we first apply the `loop-unroll` pass to unroll the loop into individual instructions (which first increases code size) before applying `instcombine`, the code size can be further optimized (optimized version 2).

In this example, the `loop-unroll` pass does not directly optimize the code size but is critical when combined with the subsequent `instcombine` pass. We refer to passes like `loop-unroll` as *assistant passes*, and pass chains like `[loop-unroll, instcombine]` as *booster pass chains* (BPCs).

3 Approach Overview

Our approach follows the generate-and-select framework, which first generates a coreset of candidate pass sequences (*generate* stage), followed by training an MLP model to select sequences from this set for a given program (*select* stage). Specifically, we focus on the first stage, as the quality of the coreset is crucial in determining the upper limit of optimization capabilities.

3.1 Generate Stage: Coreset Construction

The coreset construction is built upon the concept of booster pass chains (See Section 4.1 for formal definition). This leads to two natural questions:

- How to efficiently identify assistant passes as well as the corresponding BPCs?

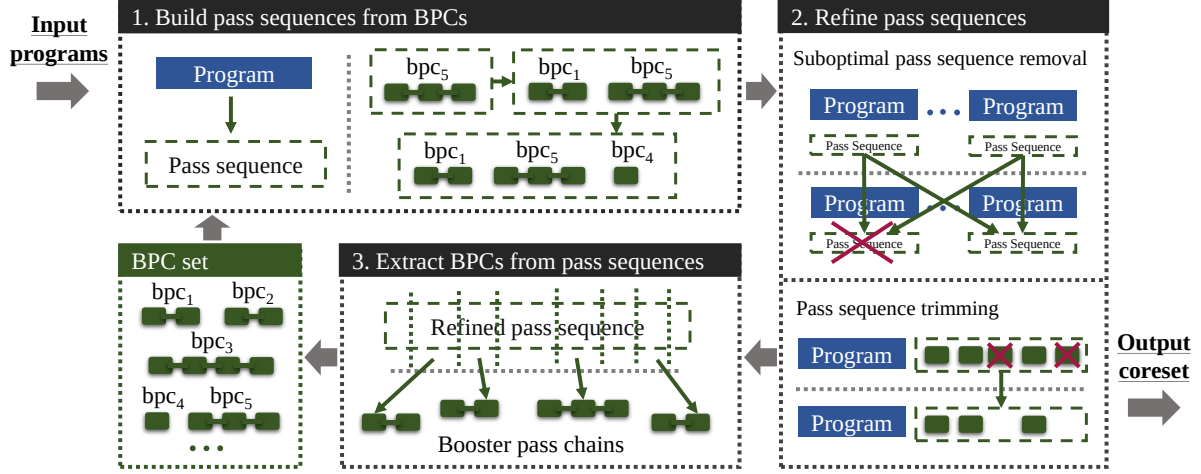


Figure 2: The overview of our coreset construction stage.

- How to effectively leverage BPCs to construct the candidate pass sequences?

We propose an iterative algorithm which simultaneously answer the above two questions to construct the coreset, as depicted in Figure 2.

- (1) *Build pass sequences from BPCs.* Given a set of input programs, we first propose an assistant-aware greedy algorithm to leverage the current BPCs to generate pass sequences, identifying assistant passes during the generation (see Section 4.2 for details).
- (2) *Refine pass sequences.* This step removes sub-optimal sequences and redundant passes. Such refinements on pass sequences are essential to produce more meaningful BPCs (see Section 4.3 for details).
- (3) *Extract BPCs from pass sequences.* Finally, rather than enumerating all possible pass combinations which is of high cost, we extract new BPCs from the refined pass sequences to update the BPC set, via a backward-scan algorithm (see Section 4.4 for details).

The BPC set maintains the current BPCs, and it is initialized with each existing compiler pass. After a few iterations when the BPC set does not update or the maximum quota has been exhausted, we select a subset of the refined pass sequences to form the coreset.

3.2 Select Stage: Sequence Prediction

For completeness, we also provide a prediction model that selects the suitable pass sequences for any program. Specifically, we train an MLP model due to its low complexity and thus high scalability. For a program, various existing feature sets can be used. We consider Autophase features [25] and InstCount features [20] because they are easy to obtain and have shown remarkable performance. The output of MLP is the predicted probability that whether a pass sequence in the coreset is the most suitable one.

4 Coreset Construction

In this section, we present the detailed approach for constructing the coreset of pass sequences.

4.1 Notations and Definitions

Throughout the paper, we use $seq = [o_1, o_2, \dots, o_l]$ to denote a pass sequence/chain, where o_i stands for a compiler pass. The code size reduction effect of applying seq on program p is defined as $r_p(seq) = s_p(\square) - s_p(seq)$, where $s_p(seq)$ is the code size obtained by applying sequence seq to program p . Based on the above notations, we formally define *assistant pass* and *booster pass chain* as follows.

Definition 4.1. For a program p and a pass chain seq , an *assistant pass* is a compiler pass o_i satisfying:

$$r_p([o_i]; seq) - r_p([o_i]) > r_p(seq). \quad (1)$$

Definition 4.2. A *booster pass chain* is a pass chain in which every compiler pass except the last is an assistant pass.

Eq. (1) essentially requires that, there exists at least one (transformed) program such that when pass o_i is applied first (left side), the code size reduction effect of seq is better than the case without o_i (the right side). In other words, each pass in a BPC enhances the optimization effect of its subsequent passes. Note that assistant passes can be overlooked in a naive search as they do not directly optimize the program. Additionally, while we keep the BPCs that are effective for a single program, we will select the subset with the highest generality among them under the coreset size constraint.

4.2 Building Pass Sequences from BPCs

To make use of the existing BPCs and provide the possibility for new BPCs, we propose an assistant-aware greedy algorithm for building pass sequences from BPCs. That is, while the naive greedy strategy selects the BPC that currently has the best effect on reducing code size, we further allow the insertion of another BPC in front of the current one, to see if further improvement can be achieved.

Algorithm 1: Pass Sequence Building Algorithm

Input : program set $P = \{p_1, p_2, \dots, p_n\}$, and BPC set $B = \{bpc_1, bpc_2, \dots, bpc_m\}$
Output: the generated pass sequences SEQ for P

```

1 Function BuildPassSequence (P, B):
2   SEQ  $\leftarrow []$ ;
3   for  $p \in P$  do
4     seq, bpc0, bpc1  $\leftarrow [], [], []$ ;
5     repeat
6       seq  $\leftarrow$  seq + bpc0 + bpc1;
7       bpc1  $\leftarrow \operatorname{argmax}_{bpc \in B} r_p(\text{seq} + bpc)$ ;
8       bpc0  $\leftarrow \operatorname{argmax}_{bpc \in B} r_p(\text{seq} + bpc + bpc_1)$ ;
9     until  $r_p(\text{seq} + bpc_0 + bpc_1) \leq r_p(\text{seq})$ ;
10    SEQ.add(seq);
11  return SEQ;
```

The assistant-aware greedy algorithm is summarized in Alg. 1. At the beginning, n programs and the current BPC set are used as input, where the BPC set is initialized with individual passes. The key difference of our greedy algorithm lies in Line 8. That is, after identifying bpc_1 which can maximally boost the optimization effect when combined with the current passes seq , we intentionally search for another BPC bpc_0 and insert it before bpc_1 to see if further code size reduction is achieved. Note that Line 8 may return null, meaning no such BPC exists. The algorithm outputs n pass sequences, one for each input program.

4.3 Refining Pass Sequences

The greedily built pass sequences from the previous step may still have quality issues, impacting the extraction of high-quality BPCs. To tackle this problem, we propose two refinement steps, i.e., *suboptimal pass sequence removal* which removes the pass sequences that do not provide the best optimization effect for any input program, and *pass sequence trimming* which removes the redundant passes that are useless (or even cause negative effect) in the corresponding pass sequences. Both steps are essential for the final coreset, as will be confirmed by our experimental results.

Suboptimal pass sequence removal. Starting with the n pass sequences from the previous step, we first remove those that do not provide the best optimization for any of the input n programs. Specifically, we apply each of the n sequences to each input program, measure code size reduction, and retain only the best sequence for each program. After this step, some input programs may share the same optimal sequence.

Pass sequence trimming. We then apply a trimming step to eliminate redundant passes in each pass sequence. Specifically, for a program p and its corresponding pass sequence seq , we consider a pass o_i in seq to be redundant if removing it from seq does not change or even increases the optimization effect.

Algorithm 2: BPC Extraction Algorithm

Input : program set $P = \{p_1, p_2, \dots, p_n\}$ and their refined pass sequences $SEQ = \{seq_1, seq_2, \dots, seq_n\}$
Output: updated BPC set B

```

1 Function ExtractBPC (P, B):
2   for  $p_i, seq_i \in P, SEQ$  do
3      $[o_1, o_2, \dots, o_l] \leftarrow seq_i$ ;
4      $y \leftarrow l$ ;
5     while  $y > 0$  do
6        $x \leftarrow y$ ;
7       while  $x > 1$  and  $o_{x-1}$  enhances
8          $[o_x, o_{x+1}, \dots, o_y]$  on program  $p'_i$  do
9            $x \leftarrow x - 1$ ;
10        B.add( $[o_x, o_{x+1}, \dots, o_y]$ );
11     $y \leftarrow x$ ;
12  return B;
```

To eliminate redundant passes, we use delta debugging [56], which aims to reduce object's size while preserving certain properties. Formally, given a set X and its subset Y , along with a test function ϕ that checks whether Y retains the desired property, delta debugging aims to find the smallest subset Y^* such that $\phi(Y^*) = \text{True}$. In our work, we adopt the Probabilistic Delta Debugging algorithm (ProbDD) [53] instead of traditional delta debugging owing to its better efficiency. Here we treat the pass sequence as the set X , and define the property as achieving the same or better optimization effect compared to X . Then, ProbDD then shortens X as much as possible while preserving or improving its code size reduction effect.

4.4 Extracting BPCs from Pass Sequences

To extract new BPCs, rather than enumerating all possible pass combinations which is of high cost, we propose a backward-scan algorithm upon the refined pass sequences. Specifically, for an input program p and its pass sequence seq , we measure the optimization effect for each pass from back to front to see if it enhances the subsequent passes (i.e., satisfying Eq. (1)). Consecutive passes satisfying the above condition are concatenated to form a BPC.

The algorithm for extracting BPCs is described in Alg. 2. We start from the second to last pass, and then move backward one by one. For each pass, we check whether it is an assistant pass, to decide whether to concatenate or split. In Line 7, o_x enhances $[o_{x+1}, o_{x+2}, \dots, o_y]$ on p' means Eq. (1) holds on the transformed program p' . In practice, since the BPC set could grow larger and thus affecting efficiency, we only keep the newly extracted BPCs to the next round.

4.5 Final Coreset Construction

After several iterations of the above three steps, we construct the coreset C based on the refined pass sequences. Our goal is to select k pass sequences from the n refined pass sequences denoted as set R , where k is predefined size of the coreset. Based on the assumption that the chosen pass sequences perform well on the input programs

Algorithm 3: Coreset Construction Algorithm

Input : program set $P = \{p_1, p_2, \dots, p_n\}$, their refined pass sequences $SEQ = \{seq_1, seq_2, \dots, seq_n\}$, and coreset size k

Output: a coreset C containing k pass sequences

```

1 Function ConstructCoreset ( $P, SEQ, k$ ) :
2    $C \leftarrow \{\}$ ;
3    $low, high \leftarrow 0, 100$ ;
4   while  $high - low > 10^{-3}$  do
5      $\alpha \leftarrow (low + high)/2$ ;
6      $current\_set \leftarrow \text{probDD}(P, SEQ, \alpha)$ ;
7     if  $|current\_set| \leq k$  then
8        $C \leftarrow current\_set$ ;
9        $low \leftarrow \alpha$ ;
10    else
11       $high \leftarrow \alpha$ ;
12  return  $C$ ;

```

will also perform well on unseen programs, the problem can be formulated as an optimization problem:

$$C = \arg \max_{C \subseteq R} \left(\prod_{p \in P} \max_{seq \in C} \frac{s_p(O_z)}{s_p(seq)} \right)^{\frac{1}{|P|}} \quad \text{s.t. } |C| = k \quad (2)$$

where $G(C) = \prod_{p \in P} \max_{seq \in C} \frac{s_p(O_z)}{s_p(seq)}^{\frac{1}{|P|}}$ is geometric mean of the maximal optimization improvement using coreset C , and we aim to maximize the geometric mean improvement over LLVM -Oz on the input programs P .

Eq. (2) is an NP-hard problem. We design an approximation algorithm combining the binary search method and the probabilistic delta debugging algorithm ProbDD [53]. First, we utilize the ProbDD algorithm to find the smallest coreset C that achieves at least $\alpha\%$ of the total performance for a given $\alpha \in [0, 100]$, i.e., $G(C) \geq G(R) \times \alpha\%$. To do this, we model the problem as follows. Let R represents a collection of n pass sequences, and C is a subset of R . Define $\phi(C) = T$ if and only if $G(C) \geq G(R) \times \alpha\%$. This allows us to directly apply ProbDD to solve the problem. Then, we know that smaller values of α require fewer pass sequences, while larger values require more. Therefore, we perform a binary search on α to find the maximum of α where the number of required pass sequences is exactly k . The complete algorithm is summarized in Alg. 3, where the `probDD()` function in Line 6 returns the smallest subset of SEQ that achieves $\alpha\%$ performance on the n input programs.

5 Sequence Prediction

Since this is not the focus of this paper, we implement a simple but effective MLP model to predict the suitable pass sequences for a program, although more complex models have been proposed (such as the graph-based model in NVP [37]). Our MLP (with a size of $56 \times 128 \times 50$) runs much faster than complex neural networks, taking

only tens of milliseconds per inference on CPU without slowing compilation.

Various static and dynamic program features have been extracted by existing work. In this work, we adopt *Autophase* features [25] which summarizing the static LLVM-IR representation, and *Inst-Count* features which count the instructions, basic blocks, functions, and 67 distinct instruction types [20], as they have shown remarkable performance. During training, we normalize the optimization effects of each pass sequence in the coreset using the softmax function, and adopt Kullback–Leibler (KL) divergence as the loss function. For prediction, our model takes program features as input and outputs a k -dimensional probability distribution, where k is the size of the coreset. A higher value in the distribution indicates a higher likelihood that the corresponding pass sequence optimizes the current program better. We rank the pass sequences from highest to lowest based on the probability distribution.

6 Evaluation

We conducted experiments using the CompilerGym [20] framework with LLVM version 10.0.0. Our evaluation aims to answer the following questions:

- **RQ1:** How is the quality of our constructed coreset in optimizing code size?
- **RQ2:** How effective is the proposed method when the selection module is included?
- **RQ3:** How does each component contribute to the overall performance?

Our experiments are performed on a cloud computing environment with 48 virtual CPUs (Intel Xeon Gold 5118 CPU @ 2.30GHz), 128G memory, and Ubuntu 22.04 operating system.

6.1 Dataset and Metric

6.1.1 Dataset. We utilize all the 14 built-in datasets from CompilerGym [20]. Additionally, we collect two datasets comprising user-submitted code from programming competition platforms Codeforces¹ and AtCoder². The CompilerGym datasets are characterized by the average number of instructions, branching instruction density, memory operation density, and arithmetic instruction density. Programs in Codeforces and AtCoder contain more header files and predefined templates, helping evaluate the proposed method in processing redundant code and complex encapsulation.

The statistics of these 16 datasets are shown in Table 1.³ For the first 10 datasets, we randomly select some programs into training, validation, and test sets; the rest 6 datasets are all treated as test sets. The training and validation sets are mainly used for training the pass sequence selection model. We further randomly select 50 programs in each training set (500 programs in total from the first 10 datasets) for our coreset construction stage (i.e., $n = 500$), and the coreset size is set to $k = 50$.

6.1.2 Metric. Following existing practices, we adopt the more general LLVM IR instruction count as an approximate to the binary

¹<https://codeforces.com/>

²<https://atcoder.jp/>

³Unlike NVP [37] which excludes programs with more than 10k IR instructions due to GEAN's large GPU memory consumption, our MLP model is more memory-efficient and thus we remove this restriction.

Table 1: Dataset statistics.

Dataset	Train	Val	Test
anghabench-v1	7,000	1,000	2,000
blas-v0	100	100	100
github-v0	7,000	1,000	1,000
linux-v0	5,000	1,000	1,000
opencv-v0	100	100	100
poj104-v1	7,000	1,000	1,000
tensorflow-v0	500	100	100
clgen-v0	300	200	200
csmith-v0	200	100	100
llvm-stress-v0	700	200	200
cbench-v1	0	0	22
chstone-v0	0	0	12
mibench-v1	0	0	40
npb-v0	0	0	122
codeforces	0	0	113
atcoder	0	0	77
Total	27,900	4,800	6,186

code size. We assess both the geometric and arithmetic means over the -Oz option.

- **GMeanOverOz.** The geometric mean over LLVM -Oz for a dataset P is defined as:

$$\bar{I}_G^{Oz} = GMeanOverOz := \left(\prod_{p \in P} \frac{s_p(Oz)}{s_p(seq)} \right)^{\frac{1}{|P|}}, \quad (3)$$

where $s_p(seq)$ is the code size after applying pass sequence seq on program p .

- **AMeanOverOz.** Similarly, the arithmetic mean over LLVM -Oz is defined as:

$$\bar{I}_A^{Oz} = AMeanOverOz := \frac{1}{|P|} \sum_{p \in P} \frac{s_p(Oz) - s_p(seq)}{s_p(Oz)}. \quad (4)$$

6.2 RQ1. Coreset Quality

6.2.1 Setup. We first evaluate the quality of the constructed coreset. We compare our BPC coreset with the coresets obtained from NVP [37], ICMC [38], and GENS [50].⁴ To ensure fairness, the search overhead of coreset construction for each method was controlled to be approximately the same scale (Table 2). The only exception was NVP-2, which is directly adopted from NVP. For each coreset, we iteratively apply all its pass sequences for a given program, and record the best result. Such results reflect the upper limit of a coreset.

(1) *BPC coreset.* This is our coreset, containing 50 pass sequences obtained from 500 programs after three rounds of iteration. The average length of pass sequences is 11.6.

(2) *NVP coreset.* Given a program, NVP [37] adopts random search strategy generating 200 random pass sequences (of length 45), and then selects the most effective one. It then repeats the above process over a set of programs, and selects 50 out of them using a

⁴We did not include the reinforcement learning based methods as they have been shown to be less effective than NVP [37].

Table 2: Runtime overhead (CPU hours) of different coreset construction methods.

BPC	NVP-1	ICMC	GENS
350 hours	370 hours	450 hours	300 hours

greedy algorithm. We compared two versions of NVP coreset, each containing 50 pass sequences:

- **NVP-1 coreset.** We use the same 500 programs with BPC, and follow NVP’s construction of coreset. Since BPC uses a smaller set of programs, we randomly search 7,000 sequences (instead of 200) for each program. The average length of each pass sequence is 21.2.
- **NVP-2 coreset:** We directly employ the coreset provided by NVP, which is obtained from a set of 17,500 programs. Each pass sequence contains 12.5 passes on average.

(3) *ICMC coreset.* ICMC [38] first constructs a graph based on the collaborative effect of two passes, generates a few pass subsequences using a graph search algorithm, and then uses them to form the final sequences for a given program in a CF (collaborative filtering) and greedy manner. Since ICMC is targeted for runtime optimization, which uses CF to reduce iterative compilation cost, we further improve it for code size optimization by exploring all possible choices instead of CF. We apply this improved ICMC method to the same 500 programs, obtaining a coreset of 50 pass sequences with our coreset generation algorithm (i.e., Alg. 3). Each pass sequence contains 62.0 passes on average.

(4) *GENS coreset.* GENS, proposed in Coverset [50], is a genetic algorithm that searches for the appropriate pass sequence for each program. We apply GENS to the same 500 programs with BPC, and follow Coverset to reduce the pass sequence length. We then also use Alg. 3 to select 50 pass sequences, denoted as the GENS coreset. Each pass sequence in the GENS coreset contains 8.8 passes on average.

6.2.2 Results. Table 3 and Table 4 show the improvements of geometric and arithmetic means, respectively, on the 6,186 test programs from the 16 datasets, where the best results are in bold. First of all, we can observe that our BPC coreset generally outperforms LLVM’s default -Oz flag, achieving 9.9% improvement in geometric mean, and 7.0% improvement in arithmetic mean on average. It also outperforms all the competitors, demonstrating the effectiveness of the proposed coreset construction approach.

Among the competitors, GENS achieves the best result (e.g., 5.4% GMeanOverOz and 4.0% AMeanOverOz), indicating the usefulness of well-established genetic algorithms in compiler optimization. NVP achieves positive results on GMeanOverOz but negative results on AMeanOverOz. This is mainly due to the fact that NVP performs significantly worse in a few programs (more discussions later), and the AMeanOverOz metric is more sensitive to such cases. ICMC also performs relatively poorly across multiple datasets. This is because ICMC adopts sharp approximations (e.g., the graph search step) in pass subsequence generation, which inevitably introduce noise into the coreset.

Table 3: Evaluation results of different coresets $\bar{I}_G^{Oz}$. Our BPC coreset outperforms LLVM -Oz and the other competitors. Best results per dataset are in bold.

Dataset	BPC	NVP-1	NVP-2	ICMC	GENS
anghabench-v1	1.010	1.006	1.009	0.974	1.008
blas-v0	1.046	1.021	1.028	1.010	1.050
github-v0	1.013	1.014	1.013	1.011	1.013
linux-v0	1.007	1.007	1.007	1.004	0.998
opencv-v0	1.124	1.108	1.109	1.060	1.107
poj104-v1	1.120	1.079	1.089	0.961	1.103
tensorflow-v0	1.081	1.070	1.077	1.029	1.075
clgen-v0	1.059	1.054	1.058	1.054	1.052
csmith-v0	1.354	0.834	0.893	1.048	1.228
llvm-stress-v0	1.057	1.014	1.079	0.934	1.050
cbench-v1	1.042	1.031	1.035	0.857	1.044
chstone-v0	1.107	1.083	1.100	0.924	1.099
mibench-v1	1.028	1.015	1.020	0.929	1.023
npb-v0	1.176	1.106	1.157	1.050	1.114
codeforces	1.278	0.855	1.185	0.905	0.932
atcoder	1.142	0.903	0.884	0.983	0.992
Average	1.099	1.009	1.043	0.982	1.054

Table 4: Evaluation results of different coresets $\bar{I}_A^{Oz}$. Our BPC coreset outperforms LLVM -Oz and the other competitors. Best results per dataset are in bold.

Dataset	BPC	NVP-1	NVP-2	ICMC	GENS
anghabench-v1	0.3%	0.2%	0.2%	-3.6%	0.2%
blas-v0	4.3%	2.0%	2.7%	0.9%	4.6%
github-v0	1.1%	1.2%	1.1%	1.0%	1.1%
linux-v0	0.7%	0.7%	0.7%	0.4%	-1.6%
opencv-v0	10.1%	9.0%	9.1%	4.9%	9.0%
poj104-v1	8.8%	5.8%	6.7%	-6.2%	7.7%
tensorflow-v0	6.9%	6.0%	6.6%	2.2%	6.4%
clgen-v0	5.3%	4.9%	5.3%	4.8%	4.8%
csmith-v0	23.3%	-73.1%	-207.0%	-5.7%	16.0%
llvm-stress-v0	3.7%	-1.2%	5.9%	-10.6%	3.0%
cbench-v1	3.7%	2.6%	3.1%	-17.2%	3.9%
chstone-v0	9.3%	7.4%	8.8%	-8.7%	8.7%
mibench-v1	2.3%	1.2%	1.7%	-7.9%	2.0%
npb-v0	10.6%	7.5%	9.7%	2.7%	8.0%
codeforces	11.7%	-17.7%	4.8%	-12.3%	-8.0%
atcoder	9.9%	-12.0%	-61.1%	-4.1%	-2.2%
Average	7.0%	-3.5%	-12.6%	-3.7%	4.0%

Considering different datasets, BPC achieves the best results in 12 out of 16 cases, especially in csmith-v0, codeforces, and atcoder when comparing with the competitors. The reason is that programs in these datasets have numerous code size optimization opportunities, amplifying the differences between methods. For example, the programs in csmith-v0 are randomly generated, while the programs in codeforces and atcoder are often of competition nature, with

Table 5: Coreset comparison between methods. BPC is better than LLVM -Oz in 43.7% programs.

	BPC	NVP-1	NVP-2	ICMC	GENS	LLVM -Oz
BPC	-	23.0%	14.8%	47.3%	19.1%	43.7%
NVP-1	3.9%	-	4.4%	44.3%	10.1%	38.2%
NVP-2	8.1%	21.9%	-	45.6%	16.3%	41.7%
ICMC	2.3%	5.3%	4.1%	-	4.7%	24.3%
GENS	5.4%	20.3%	10.5%	44.8%	-	40.7%
LLVM -Oz	7.6%	13.4%	9.8%	32.8%	10.1%	-

little attention paid to the program optimization aspect. We further manually analyzed these datasets and found that the superior performance of the BPC coreset is mainly due to the synergistic effect between passes (i.e., the booster pass chain). A case study is presented in Section 6.5.

Table 5 further compares the optimization effects of different methods by program. Specifically, each entry in the table shows the percentage of programs that one method outperforms the other. For example, 23.0% in the first row means BPC is better than NVP-1 in 23.0% programs. Notably, our method exceeds LLVM -Oz in 43.7% programs, the highest rate among all methods, and underperforms in only 7.6%, the lowest rate. This suggests our method’s broad applicability and effectiveness across various programs.

6.3 RQ2. Selection Effectiveness

6.3.1 Setup. To address RQ2, we train an MLP to predict top- K pass sequences in the coreset for each test program. We conduct experiments on BPC, NVP-1, NVP-2, and GENS coresets, and exclude ICMC coreset as it performs relatively poorly in the previous experiment. We evaluate only GMeanOverOz (robust to outliers) because our focus is the model’s capability, disregarding coreset quality, thus excluding AMeanOverOz. In terms of the features used in training the MLP, we consider the following options:

- *Autophase* feature [25]. The Autophase feature is a 56-dimension integer feature vector that summarizes the LLVM-IR features of a program. It includes both instruction count statistics and specific control flow information statistics (e.g., number of critical edges).
- *InstCount* feature [20]. The InstCount feature of a program is a 70-dimension integer feature vector. It includes the count of various types of instructions within the program.

Both feature sets are static features which are easy to obtain. For each set of features, we design an MLP structure with one hidden layer of 128 neurons.

6.3.2 Results. The results are presented in Figure 3. The x-axis represents the top- K predicted pass sequences from the coreset. The y-axis is the GMeanOverOz metric with the best prediction kept. It can be observed that the performance of our approach (on both feature sets) remains superior to the competitors. For example, when predicting top-3 pass sequences with the InstCount feature set, the GMeanOverOz metric is 1.069. When top-10 pass sequences are selected, the GMeanOverOz metric becomes 1.092. This result also

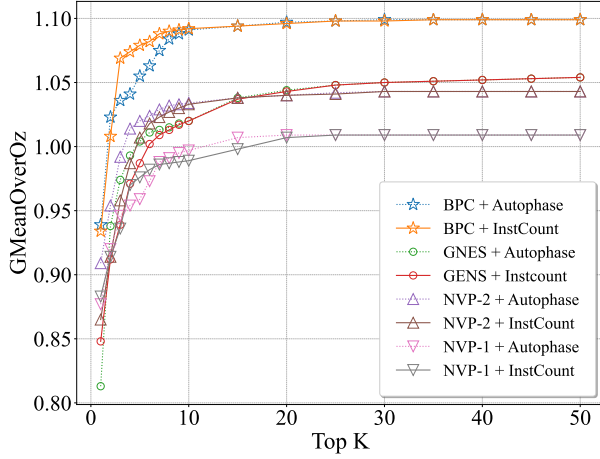


Figure 3: Evaluation results when top- K predictions from coreset are used.

Table 6: Ablation results on the building step and refinement step in coreset construction. Both them are helpful.

Method	GMeanOverOz	Avg. Length
full	1.099	11.60
- Assistant-aware	1.045 (-5.4%)	9.10
- Removal	1.070 (-2.9%)	10.32
- Trimming	1.082 (-1.7%)	14.72

indicates that with an appropriate representation of program characteristics, a simple MLP model is capable of effectively predicting suitable pass sequences from the coreset for specific programs.

6.4 RQ3. Ablation Study

To address RQ3, we evaluate the impact of the building step (assistant-aware greedy algorithm), refinement step and the coreset construction algorithm. We did not conduct ablation experiments on the BPC extraction steps because the absence of either would render the iterative algorithm inoperative.

6.4.1 Results of Building and Refinement. For the building step, we replace the assistant-aware greedy algorithm with naive greedy algorithm. For the refinement step, we separately remove its two substeps, i.e., suboptimal pass sequence removal and pass sequence trimming. Then, we test these approach on the test set.

The results are shown in Table 6, where “full” indicates the complete algorithm; “-Assistant-aware” denote the iterative algorithms with naive greedy algorithm; “-Removal” and “-Trimming” denote the iterative algorithms with the above two steps removed, respectively. It can be observed that all of them are helpful to improve the optimization effect of the iterative algorithm. This also confirms with our insights that: (1) assistant-aware greedy algorithm could better identify assistant passes as well as the corresponding BPCs; (2) refinements serve as a denoising step for the later BPC extraction, and thus can improve the performance in the long run.

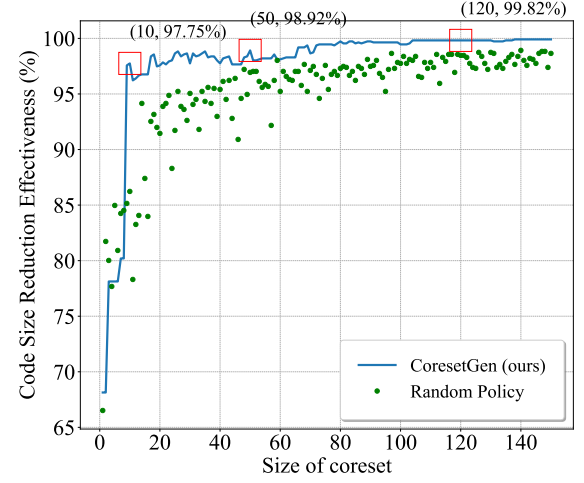


Figure 4: Ablation results of the final coreset construction algorithm.

Notably, even when either removal or trimming is removed, the resulting performance is still better than the competitors. We also observe that, removing the trimming step results in an increased number of passes, which may lead to longer compilation time.

6.4.2 Results of Alg. 3. We next compare the coreset construction algorithm proposed in Alg. 3 with a method that randomly selects 50 pass sequences from 500. The results are shown in Figure 4, where the x-axis indicates the size of the coreset and the y-axis shows the percentage of the GMeanOverOz achieved by the selected coreset relative to the overall GMeanOverOz when 500 pass sequences are all included. It can be seen that our algorithm is more effective in selecting pass sequences compared to the random method, achieving 97.75% effectiveness by selecting only 10 out of 500 pass sequences. The curve of our algorithm exhibits some small fluctuations. This is because the algorithm is optimized on the 500 input programs, which does not necessarily transfer to optimal performance on the test set. Additionally, we observe that even random selection also yields acceptable results. For example, when the coreset size is over 60, it achieves over 95% performance. The reason is that most pass sequences in our coreset have good optimization effects across different programs.

6.5 Case Study

To verify the intrinsic mechanism of our approach - specifically, whether the observed performance gains in our pass sequences result from synergistic interactions between passes - we present some BPCs extracted by our approach. The complete BPC set is included in our project website.

Case: [loop-unroll, gvn, inline, globalopt] Figure 5 illustrates the effect of this BPC on the Csmith program *csmith-v0/3043438051*, with blue rectangles indicating IR instruction counts. ‘BPC original version’ indicates the result of applying the best-performing pass sequence from our BPC coreset. Although the first three passes (loop-unroll, gvn, and inline) increase code size, combining them together with globalopt could reduce the code size to 70. In contrast,

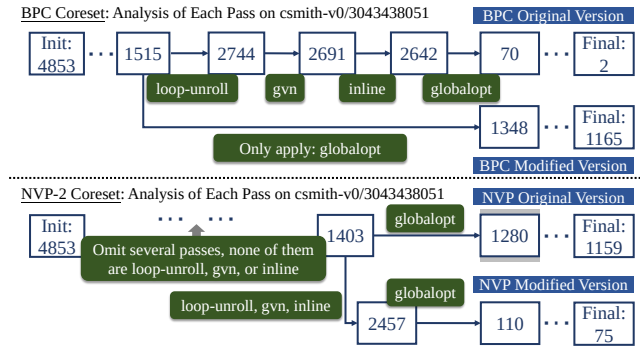


Figure 5: Case study of a Csmith program. The extracted BPC [loop-unroll, gvn, inline, globalopt] plays critical roles in optimizing this program.

applying `globalopt` alone can only reduce the code size from 1515 to 1348. Neither NVP-1 nor NVP-2 coresets include such pass chains, and thus fail to optimize the program effectively. The ‘NVP original version’ stands for NVP-2 coreset, which contains the pass `globalopt` with marginal optimization effect (from 1403 to 1280). However, replacing [globalopt] with [loop-unroll, gvn, inline, globalopt] reduce code size to 110.

7 Discussion

The present work focuses solely on code size and employs LLVM 10 (an older version, while the latest release is LLVM 20). Future studies could conduct more comprehensive experiments with updated toolchains.

Additionally, this work provides insights for future work, such as: (1) the extracted BPC set and the constructed coreset can be directly used by existing work. For example, reinforcement learning based methods can include our BPC set into the action space, where the next action could be a pass chain instead of a single pass; (2) the design of the proposed coreset construction approach is loosely coupled and highly flexible, allowing easy replacement with existing methods. For example, we may consider frequent itemset mining algorithms to replace the BPC extraction step.

8 Related Work

In this section, we briefly review the existing work. We mainly cover the iterative compilation techniques and machine learning based techniques, which are most related to our work, in the following.

8.1 Iterative Compilation

Iterative compilation is one of the mostly adopted techniques to explore the pass sequence space in compiler optimization. It iteratively applies different pass sequences to optimize the program and selects the best-performing one. Various methods have been proposed to explore the pass sequence space, including greedy methods [28], graph-based methods [44], genetic algorithms [15, 16, 31, 32], the combinations of these methods [2, 46], etc. Additionally, various techniques have been developed to reduce the search space [1, 26, 30, 42].

For example, Cooper et al. [15] introduce crossover and mutation operations in genetic algorithms to construct pass sequences. Huang et al. [28] greedily select passes for building pass sequence. Agakov et al. [1] employ machine learning to automatically search in those areas likely to achieve the best performance. Cobayn [8] uses Bayesian networks to select the passes that might improve application performance. Martins et al. [42] use clustering algorithms to identify a subset of passes, and conduct phase ordering on this subset. Recently, Silva et al. [50] adopt the generate-and-select framework, where their coreset, referred to as a *coreset*, is constructed using a genetic algorithm, and the program similarity is measured by Euclidean distance between program feature vectors. Additionally, frameworks supporting iterative compilation [4, 23, 45] have also been proposed.

Our approach also adopts the generate-and-select framework, and applies iterative compilation in coreset construction. Different from the existing work, We define the BPC rather than the single pass as the basic unit.

8.2 Machine Learning Meets Compiler Optimization

Recently, machine learning techniques have been widely adopted in compiler optimization. Here, we briefly discuss the existing program feature extraction methods and prediction strategies. More detailed information can be found in recent surveys [7, 35, 54].

Program Feature Extraction. Various features have been used to represent a program, serving as the input for machine learning models. Depending on whether requiring program execution, existing features can be divided into static features [1, 8, 33, 34, 36] and dynamic features [11, 27, 39, 41, 51]. Considering the form of features, existing features include fixed-length numerical vectors [20, 25, 43], sequences of numerical vectors [9], and graph-based features [17, 22, 47].

Prediction Strategy. In terms of prediction strategy, many machine learning methods have been applied including neural networks [37], Bayesian networks [8], SVM [5, 48], reinforcement learning (RL) [3, 25, 29, 40, 49], and large language models [19]. Some of these methods are predictive [5, 6, 12, 14, 37, 38, 48, 57], where they predict one or several pass sequences for a given program. For example, NVP [37] first obtains a set of pass sequences through random search, and then trains a graph neural network to predict pass sequences. Another way is to compute the program similarity and recommend the pass sequences from similar programs [5, 12]. Other methods are generative, where pass sequences are generated from scratch for each program. For example, Haj-Ali et al. [25] using RL-based methods, treating all passes as the action space and the compilation result as feedback.

Since this is not the focus of the paper, we simply reuse existing feature sets and adopt an MLP structure for prediction. Other techniques such as the E-graph solution in the HLS domain [13] are also not covered.

9 Conclusion

In this paper, we propose a new approach for the phase ordering problem in compiler optimization. The key of our approach is the concept of booster pass chain, whose presence as a whole is useful

to optimize certain programs. Following the generate-and-select framework, we then propose an iterative algorithm that automatically identifies and utilizes booster pass chains, resulting in a set of BPCs as well as a coreset of candidate pass sequences. A neural network model is also trained in the select stage. We test our approach on 6,186 unseen programs across 16 datasets, and the results show its superior performance in code size reduction, achieving 9.9% improvement over LLVM -Oz. In the future, we plan to extend the proposed approach in optimizing other targets such as execution time, other than code size. We are also interested in testing the identified BPC set and coreset by plugging them into the existing approaches.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grants #62025202), the Frontier Technologies R&D Program of Jiangsu (BF2024059), and the Collaborative Innovation Center of Novel Software Technology and Industrialization. Yuan Yao is the corresponding author.

References

- [1] Felix Agakov, Edwin Bonilla, John Cavazos, Björn Franke, Grigori Fursin, Michael FP O'Boyle, John Thomson, Marc Toussaint, and Christopher KI Williams. 2006. Using machine learning to focus iterative optimization. In *International Symposium on Code Generation and Optimization (CGO'06)*. IEEE, 11–pp.
- [2] Lelac Almagor, Keith D Cooper, Alexander Grosul, Timothy J Harvey, Steven W Reeves, Devika Subramanian, Linda Torczon, and Todd Waterman. 2004. Finding effective compilation sequences. *ACM SIGPLAN Notices* 39, 7 (2004), 231–239.
- [3] Mohammed Almakki, Ayman Izzeldin, Qijing Huang, Ameer Haj Ali, and Chris Cummins. 2022. Autophase V2: Towards Function Level Phase Ordering Optimization. In *Machine Learning for Computer Architecture and Systems 2022*.
- [4] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. Opentuner: An extensible framework for program autotuning. In *Proceedings of the 23rd international conference on Parallel architectures and compilation*. 303–316.
- [5] Amir Hossein Ashouri, Andrea Bignoli, Gianluca Palermo, and Cristina Silvano. 2016. Predictive modeling methodology for compiler phase-ordering. In *Proceedings of the 7th workshop on parallel programming and run-time management techniques for many-core architectures and the 5th workshop on design tools and architectures for multicore embedded computing platforms*. 7–12.
- [6] Amir H Ashouri, Andrea Bignoli, Gianluca Palermo, Cristina Silvano, Sameer Kulkarni, and John Cavazos. 2017. Micomp: Mitigating the compiler phase-ordering problem using optimization sub-sequences and machine learning. *ACM Transactions on Architecture and Code Optimization (TACO)* 14, 3 (2017), 1–28.
- [7] Amir H Ashouri, William Killian, John Cavazos, Gianluca Palermo, and Cristina Silvano. 2018. A survey on compiler autotuning using machine learning. *ACM Computing Surveys (CSUR)* 51, 5 (2018), 1–42.
- [8] Amir Hossein Ashouri, Giovanni Mariani, Gianluca Palermo, Eunjung Park, John Cavazos, and Cristina Silvano. 2016. Cobayn: Compiler autotuning framework using bayesian networks. *ACM Transactions on Architecture and Code Optimization (TACO)* 13, 2 (2016), 1–25.
- [9] Tal Ben-Nun, Alice Shoshana Jakobovits, and Torsten Hoefler. 2018. Neural code comprehension: A learnable representation of code semantics. *Advances in neural information processing systems* 31 (2018).
- [10] François Bodin, Toru Kisuki, Peter Knijnenburg, Mike O'Boyle, and Erven Rohou. 1998. Iterative compilation in a non-linear optimisation space. In *Workshop on profile and feedback-directed compilation*.
- [11] John Cavazos, Grigori Fursin, Felix Agakov, Edwin Bonilla, Michael FP O'Boyle, and Olivier Temam. 2007. Rapidly selecting good compiler optimizations using performance counters. In *International Symposium on Code Generation and Optimization (CGO'07)*. IEEE, 185–197.
- [12] Stefano Cereda, Gianluca Palermo, Paolo Cremonesi, and Stefano Doni. 2020. A collaborative filtering approach for the automatic tuning of compiler optimisations. In *The 21st ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems*. 15–25.
- [13] Jianyi Cheng, Samuel Coward, Lorenzo Chelini, Rafael Barbalho, and Theo Drane. 2024. SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1029–1044.
- [14] Keith D Cooper, Alexander Grosul, Timothy J Harvey, Steve Reeves, Devika Subramanian, Linda Torczon, and Todd Waterman. 2006. Exploring the structure of the space of compilation sequences using randomized search algorithms. *The Journal of Supercomputing* 36 (2006), 135–151.
- [15] Keith D Cooper, Philip J Schielke, and Devika Subramanian. 1999. Optimizing for reduced code space using genetic algorithms. In *Proceedings of the ACM SIGPLAN 1999 workshop on Languages, compilers, and tools for embedded systems*. 1–9.
- [16] Keith D Cooper, Devika Subramanian, and Linda Torczon. 2002. Adaptive optimizing compilers for the 21st century. *The Journal of Supercomputing* 23 (2002), 7–22.
- [17] Chris Cummins, Zacharias V Fisches, Tal Ben-Nun, Torsten Hoefler, Michael FP O'Boyle, and Hugh Leather. 2021. Programl: A graph-based program representation for data flow analysis and compiler optimizations. In *International Conference on Machine Learning*. PMLR, 2244–2253.
- [18] Chris Cummins, Pavlos Petoumenos, Zheng Wang, and Hugh Leather. 2017. Synthesizing benchmarks for predictive modeling. In *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 86–99.
- [19] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, et al. 2023. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062* (2023).
- [20] Chris Cummins, Bram Wasti, Jiadong Guo, Brandon Cui, Jason Ansel, Sahir Gomez, Somya Jain, Jia Liu, Olivier Teytaud, Benoit Steiner, et al. 2022. Compiler-robust: Robust, performant compiler optimization environments for ai research. In *2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 92–105.
- [21] Anderson Faustino Da Silva, Bruno Conde Kind, José Wesley de Souza Magalhães, Jerônimo Nunes Rocha, Breno Campos Ferreira Guimaraes, and Fernando Magno Quinão Pereira. 2021. Anghabench: A suite with one million compilable c benchmarks for code-size reduction. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 378–390.
- [22] Jeanne Ferrante, Karl J Ottenstein, and Joe D Warren. 1987. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 9, 3 (1987), 319–349.
- [23] Grigori Fursin, Renato Miceli, Anton Lokhmotov, Michael Gerndt, Marc Baboulin, Allen D Malony, Zbigniew Chamski, Diego Novillo, and Davide Del Vento. 2014. Collective mind: Towards practical and collaborative auto-tuning. *Scientific Programming* 22, 4 (2014), 309–329.
- [24] Matthew R Guthaus, Jeffrey S Ringenberg, Dan Ernst, Todd M Austin, Trevor Mudge, and Richard B Brown. 2001. MiBench: A free, commercially representative embedded benchmark suite. In *Proceedings of the fourth annual IEEE international workshop on workload characterization. WWC-4 (Cat. No. 01EX538)*. IEEE, 3–14.
- [25] Ameer Haj-Ali, Qijing Jenny Huang, John Xiang, William Moses, Krste Asanovic, John Wawrzyniec, and Ion Stoica. 2020. Autophase: Juggling his phase orderings in random forests with deep reinforcement learning. *Proceedings of Machine Learning and Systems* 2 (2020), 70–81.
- [26] Ruobing Han and Hyesoon Kim. 2024. Exponentially Expanding the Phase-Ordering Search Space via Dormant Information. In *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*. 250–261.
- [27] Kenneth Hoste and Lieven Eeckhout. 2007. Microarchitecture-independent workload characterization. *IEEE micro* 27, 3 (2007), 63–72.
- [28] Qijing Huang, Ruolong Lian, Andrew Canis, Jongsok Choi, Ryan Xi, Stephen Brown, and Jason Anderson. 2013. The effect of compiler optimizations on high-level synthesis for FPGAs. In *2013 IEEE 21st Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, 89–96.
- [29] Shalini Jain, Yashas Andaluri, S VenkataKeerthy, and Ramakrishna Upadrastra. 2022. Poset-rl: Phase ordering for optimizing size and execution time using reinforcement learning. In *2022 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 121–131.
- [30] Michael R Jantz and Prasad A Kulkarni. 2013. Exploiting phase inter-dependencies for faster iterative compiler optimization phase order searches. In *2013 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*. IEEE, 1–10.
- [31] Prasad Kulkarni, Stephen Hines, Jason Hiser, David Whalley, Jack Davidson, and Douglas Jones. 2004. Fast searches for effective optimization phase sequences. *ACM SIGPLAN Notices* 39, 6 (2004), 171–182.
- [32] Prasad Kulkarni, Wankang Zhao, Hwashin Moon, Kyunghwan Cho, David Whalley, Jack Davidson, Mark Bailey, Yunheung Paek, and Kyle Gallivan. 2003. Finding effective optimization phase sequences. *ACM SIGPLAN Notices* 38, 7 (2003), 12–23.
- [33] Sameer Kulkarni and John Cavazos. 2012. Mitigating the compiler optimization phase-ordering problem using machine learning. In *Proceedings of the ACM international conference on Object oriented programming systems languages and applications*. 147–162.
- [34] Hugh Leather, Edwin Bonilla, and Michael O'boyle. 2014. Automatic feature generation for machine learning-based optimising compilation. *ACM Transactions on Architecture and Code Optimization (TACO)* 11, 1 (2014), 1–32.

- [35] Hugh Leather and Chris Cummins. 2020. Machine learning in compilers: Past, present and future. In *2020 Forum for Specification and Design Languages (FDL)*. IEEE, 1–8.
- [36] Fengqian Li, Feilong Tang, and Yao Shen. 2014. Feature mining for machine learning based compilation optimization. In *2014 Eighth International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*. IEEE, 207–214.
- [37] Youwei Liang, Kevin Stone, Ali Shameli, Chris Cummins, Mostafa Elhoushi, Jiadong Guo, Benoit Steiner, Xiaomeng Yang, Pengtao Xie, Hugh James Leather, et al. 2023. Learning compiler pass orders using coreset and normalized value prediction. In *International Conference on Machine Learning*. PMLR, 20746–20762.
- [38] Hongzhi Liu, Jie Luo, Ying Li, and Zhonghai Wu. 2021. Iterative compilation optimization based on metric learning and collaborative filtering. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 1 (2021), 1–25.
- [39] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. *Acm sigplan notices* 40, 6 (2005), 190–200.
- [40] Rahim Mammadli, Ali Jannesari, and Felix Wolf. 2020. Static neural compiler optimization via deep reinforcement learning. In *2020 IEEE/ACM 6th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC) and Workshop on Hierarchical Parallelism for Exascale Computing (HiPar)*. IEEE, 1–11.
- [41] Jason Mars and Robert Hundt. 2009. Scenario based optimization: A framework for statically enabling online optimizations. In *2009 International Symposium on Code Generation and Optimization*. IEEE, 169–179.
- [42] Luiz GA Martins, Ricardo Nobre, Joao MP Cardoso, Alexandre CB Delbem, and Eduardo Marques. 2016. Clustering-based selection for the exploration of compiler optimization sequences. *ACM Transactions on Architecture and Code Optimization (TACO)* 13, 1 (2016), 1–28.
- [43] Mircea Namolaru, Albert Cohen, Grigori Fursin, Ayal Zaks, and Ari Freund. 2010. Practical aggregation of semantical program properties for machine learning based optimization. In *Proceedings of the 2010 international conference on Compilers, architectures and synthesis for embedded systems*. 197–206.
- [44] Ricardo Nobre, Luiz GA Martins, and João MP Cardoso. 2016. A graph-based iterative compiler pass selection and phase ordering approach. *ACM SIGPLAN Notices* 51, 5 (2016), 21–30.
- [45] Cedric Nugteren and Valeriu Codreanu. 2015. CLTune: A generic auto-tuner for OpenCL kernels. In *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*. IEEE, 195–202.
- [46] Zhelong Pan and Rudolf Eigenmann. 2006. Fast and effective orchestration of compiler optimizations for automatic performance tuning. In *International Symposium on Code Generation and Optimization (CGO'06)*. IEEE, 12–pp.
- [47] Eunjung Park, John Cavazos, and Marco A Alvarez. 2012. Using graph-based program characterization for predictive modeling. In *Proceedings of the Tenth International Symposium on Code Generation and Optimization*. 196–206.
- [48] Eunjung Park, Sameer Kulkarni, and John Cavazos. 2011. An evaluation of different modeling techniques for iterative compilation. In *Proceedings of the 14th international conference on Compilers, architectures and synthesis for embedded systems*. 65–74.
- [49] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. Compiler optimization for quantum computing using reinforcement learning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [50] Anderson Faustino da Silva, Bernardo NB De Lima, and Fernando Magno Quintão Pereira. 2021. Exploring the space of optimization sequences for code-size reduction: insights and tools. In *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction*. 47–58.
- [51] Spyridon Triantafyllis, Manish Vachharajani, Neil Vachharajani, and David I August. 2003. Compiler optimization-space exploration. In *International Symposium on Code Generation and Optimization, 2003. CGO 2003*. IEEE, 204–215.
- [52] Steven R Vegdahl. 1982. Phase coupling and constant generation in an optimizing microcode compiler. *ACM SIGMICRO Newsletter* 13, 4 (1982), 125–133.
- [53] Guancheng Wang, Ruobing Shen, Junjie Chen, Yingfei Xiong, and Lu Zhang. 2021. Probabilistic delta debugging. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 881–892.
- [54] Zheng Wang and Michael O'Boyle. 2018. Machine learning in compiler optimization. *Proc. IEEE* 106, 11 (2018), 1879–1901.
- [55] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*. 283–294.
- [56] Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on software engineering* 28, 2 (2002), 183–200.
- [57] Mingxuan Zhu, Dan Hao, and Junjie Chen. 2024. Compiler Autotuning through Multiple Phase Learning. *ACM Transactions on Software Engineering and Methodology* (2024).